I²C Protocol

Engineering Handbook

STM32F4 • ESP32 • S32K1 • AUTOSAR • KV5x

Comprehensive reference with hardware overview, key register summaries, full bit-level definitions, driver examples, and AUTOSAR ComStack

1. I²C Protocol Fundamentals

1.1 Overview

Inter-Integrated Circuit (I²C) is a synchronous, half-duplex, multi-master serial bus protocol developed by Philips Semiconductor (now NXP) in 1982. It uses only two wires — SDA (Serial Data) and SCL (Serial Clock) — both of which are open-drain with pull-up resistors. This makes I²C ideal for connecting multiple low-speed peripherals (sensors, EEPROMs, RTCs, DACs, displays) to a microcontroller with minimal pin usage.

Key characteristics of I²C:

- Multi-master, multi-slave — any node may become a bus master; arbitration is handled automatically.
- 7-bit or 10-bit device addressing — supports up to 112 unique devices on one bus in 7-bit mode (16 addresses reserved).
- Acknowledged transfers — every byte is followed by an ACK or NACK from the receiver, providing built-in error detection.
- Four speed grades — Standard (100 kHz), Fast (400 kHz), Fast-Plus (1 MHz), High-Speed (3.4 MHz).
- Clock stretching — a slave may hold SCL low to pause communication until it is ready.
- No chip-select pins — devices are identified purely by their 7-bit or 10-bit address.

1.2 Physical Layer

Both SDA and SCL are open-drain lines pulled up to Vcc through external resistors. The bus is idle (both lines high) when no transfer is in progress. Pull-up resistor values affect both rise time and maximum speed:

Speed Mode	Max SCL Freq	Typical Pull-up (3.3 V bus)	Max Bus Capacitance	Rise Time (t_r max)
Standard-Mode (SM)	100 kHz	4.7 kΩ	400 pF	1000 ns
Fast-Mode (FM)	400 kHz	2.2 kΩ	400 pF	300 ns
Fast-Mode Plus (FM+)	1 MHz	1.0 kΩ	550 pF	120 ns
High-Speed Mode (HS)	3.4 MHz	~100–500 Ω	100 pF	80 ns

> *Pull-up resistors must be sized for the bus capacitance and speed. Too large causes slow rise times and bit errors; too small causes excessive current and may violate I/O sink limits.*

1.3 Bus Conditions and Transfer Sequence

I²C communication is defined by specific bus conditions:

Condition	SDA	SCL	Description

START (S)	High → Low	High	SDA falls while SCL is high. Initiated by master to claim the bus.
Repeated START (Sr)	High → Low	High	Another START without a preceding STOP. Used to change direction or address without releasing the bus.
STOP (P)	Low → High	High	SDA rises while SCL is high. Releases the bus to idle state.
Data Bit	Stable	High pulse	SDA must be stable while SCL is high. SDA changes only when SCL is low.
ACK	Low	High pulse	Receiver pulls SDA low during the 9th clock pulse to acknowledge receipt of a byte.
NACK	High	High pulse	Receiver leaves SDA high during 9th clock pulse. Signals error, end of data, or address not recognised.

A complete I²C transaction follows this structure:

- START condition
- Address byte: 7-bit slave address + R/W bit (0=write, 1=read)
- ACK from slave (if address recognised)
- Data byte(s), each followed by ACK/NACK
- STOP condition (or Repeated START for combined transactions)

1.4 7-bit vs 10-bit Addressing

Feature	7-bit Addressing	10-bit Addressing
Address space	128 (112 usable)	1024 (1008 usable)
Address byte(s)	1 byte (7-bit + R/W)	2 bytes: 11110xx0 + 8 lower bits; read uses 11110xx1
Backward compat.	Yes — default	Yes — uses reserved 11110xx prefix
Common usage	Most I²C devices	Large systems, address conflicts

1.5 Clock Stretching and Multi-Master Arbitration

Clock Stretching: A slave device may hold SCL low after the master releases it, effectively pausing the transfer. The master must detect this and wait until SCL is released before continuing. Not all master implementations support clock stretching — check hardware capability.

Arbitration: When two masters simultaneously initiate a START, each monitors SDA while transmitting. A master that transmits a recessive '1' but observes a dominant '0' has lost arbitration and must immediately stop transmitting. The winning master continues as if nothing happened. This is non-destructive — no data is corrupted.

1.6 I²C vs Other Protocols

Feature	I²C	SPI	UART	CAN
Wires	2 (SDA, SCL)	4 (MOSI, MISO, SCLK, CS)	2 (TX, RX)	2 (CAN_H, CAN_L)
Topology	Multi-master bus	Master-slave (dedicated CS)	Point-to-point	Multi-master bus
Max Speed	3.4 MHz (HS)	50+ MHz	~20 Mbps async	1 Mbit/s (classic)
Full-Duplex	No (half-duplex)	Yes	Yes	No (half-duplex)
Addressing	7/10-bit address	CS pin per slave	N/A	Message ID
ACK/NACK	Yes — per byte	No	Optional (SW)	Yes — per frame
Typical use	Sensors, EEPROMs	High-speed flash, ADC	Debug, GNSS	Automotive networks

2. I²C on STM32F4 (I2C / FMPI2C)

2.1 Hardware Overview

STM32F4 devices provide up to 3 standard I2C peripherals (I2C1–I2C3) on APB1, plus the FMPI2C1 (Fast-Mode Plus I²C) on select devices (e.g., STM32F412, STM32F413). The standard I2C peripheral supports Standard and Fast modes. FMPI2C1 additionally supports Fast-Mode Plus (1 MHz) and is register-compatible with STM32F0/F3 I2C.

Key features of STM32F4 I2C:

- Master transmitter, master receiver, slave transmitter, slave receiver modes.
- DMA support for both TX and RX.
- Analog and digital noise filters on SCL/SDA.
- SMBus (System Management Bus) compatibility.
- Clock stretching supported in both master and slave modes.
- 7-bit and 10-bit slave addressing.

2.2 Key Registers — Summary

Register	Name	Key Bits
I2C_CR1	Control Register 1	SWRST, ALERT, PEC, POS, ACK, STOP, START, NOSTRETCH, ENGC, ENPEC, ENARP, SMBTYPE, SMBUS, PE
I2C_CR2	Control Register 2	LAST, DMAEN, ITBUFEN, ITEVTEN, ITERREN, FREQ[5:0]
I2C_OAR1	Own Address 1	ADDMODE (10-bit), ADD[9:0] or ADD[7:1]
I2C_OAR2	Own Address 2	ENDUAL, ADD2[7:1] — second address in dual-address mode
I2C_DR	Data Register	DR[7:0] — write to transmit, read to receive
I2C_SR1	Status Register 1	SMBALERT, TIMEOUT, PECERR, OVR, AF, ARLO, BERR, TXE, RXNE, STOPF, ADD10, BTF, ADDR, SB
I2C_SR2	Status Register 2	PEC[7:0], DUALF, SMBHOST, SMBDEFAULT, GENCALL, TRA, BUSY, MSL
I2C_CCR	Clock Control Register	FS (Fast/Standard), DUTY, CCR[11:0]
I2C_TRISE	Rise Time Register	TRISE[5:0] — max SCL rise time in APB cycles
I2C_FLTR	Filter Register	ANOFF (analog filter), DNF[3:0] (digital noise filter)

2.3 Clock Configuration (CCR)

The SCL frequency is derived from the APB1 clock via the CCR register:

```
/* Standard Mode (Sm): CCR = t_HIGH / t_APB1
   t_HIGH = t_LOW = 1/(2 * f_SCL)
   CCR = f_APB1 / (2 * f_SCL)
   Example: f_APB1=42MHz, f_SCL=100kHz => CCR = 42000000/(2*100000) = 210

/* Fast Mode (Fm), DUTY=0 (t_LOW/t_HIGH = 2):
   CCR = f_APB1 / (3 * f_SCL)
   Example: f_APB1=42MHz, f_SCL=400kHz => CCR = 42000000/(3*400000) = 35

/* Fast Mode (Fm), DUTY=1 (t_LOW/t_HIGH = 16/9, for lower ripple):
   CCR = f_APB1 / (25 * f_SCL)
   Example: f_APB1=42MHz, f_SCL=400kHz => CCR = 42000000/(25*400000) = 4

/* TRISE for Standard Mode: TRISE = (1000 ns * f_APB1 / 1e9) + 1
   Example: f_APB1=42MHz => TRISE = (1000*42/1000) + 1 = 43
/* TRISE for Fast Mode:   TRISE = (300 ns  * f_APB1 / 1e9) + 1
   Example: f_APB1=42MHz => TRISE = (300*42/1000)  + 1 = 14 */
```

2.4 HAL Initialization and Usage

```
I2C_HandleTypeDef hi2c1;

void MX_I2C1_Init(void) {
    hi2c1.Instance              = I2C1;
    hi2c1.Init.ClockSpeed       = 400000;          /* 400 kHz Fast Mode */
    hi2c1.Init.DutyCycle        = I2C_DUTYCYCLE_2; /* t_LOW/t_HIGH = 2  */
    hi2c1.Init.OwnAddress1      = 0x00;            /* master: unused    */
    hi2c1.Init.AddressingMode   = I2C_ADDRESSINGMODE_7BIT;
    hi2c1.Init.DualAddressMode  = I2C_DUALADDRESS_DISABLE;
    hi2c1.Init.GeneralCallMode  = I2C_GENERALCALL_DISABLE;
    hi2c1.Init.NoStretchMode    = I2C_NOSTRETCH_DISABLE;
    HAL_I2C_Init(&hi2c1);
}

/* Write 2 bytes to slave at address 0x48 */
uint8_t txBuf[2] = { 0x01, 0xFF };  /* reg addr, data */
HAL_I2C_Master_Transmit(&hi2c1, 0x48 << 1, txBuf, 2, HAL_MAX_DELAY);

/* Read 2 bytes from slave at address 0x48 */
uint8_t rxBuf[2];
HAL_I2C_Master_Receive(&hi2c1, (0x48 << 1) | 0x01, rxBuf, 2, HAL_MAX_DELAY);

/* Combined write-then-read (register read pattern) */
uint8_t regAddr = 0x00;
HAL_I2C_Master_Transmit(&hi2c1, 0x48 << 1, &regAddr, 1, HAL_MAX_DELAY);
HAL_I2C_Master_Receive (&hi2c1, (0x48<<1)|1, rxBuf, 2, HAL_MAX_DELAY);

/* Memory (register) interface — single call combined Tx+Rx */
HAL_I2C_Mem_Read(&hi2c1, 0x48 << 1, 0x00, I2C_MEMADD_SIZE_8BIT,
                 rxBuf, 2, HAL_MAX_DELAY);
```

2.5 DMA-Mode Example

```
/* Non-blocking DMA transmit */
HAL_I2C_Master_Transmit_DMA(&hi2c1, 0x48 << 1, txBuf, 2);

/* Completion callback */
void HAL_I2C_MasterTxCpltCallback(I2C_HandleTypeDef *hi2c) {
    if (hi2c->Instance == I2C1) {
        /* Transmit complete — start next operation */
    }
```

```
}

/* Interrupt-driven receive */
HAL_I2C_Master_Receive_IT(&hi2c1, (0x48<<1)|1, rxBuf, 2);

void HAL_I2C_MasterRxCpltCallback(I2C_HandleTypeDef *hi2c) {
    if (hi2c->Instance == I2C1) {
        /* Process rxBuf[0..1] */
    }
}
```

2.6 Direct Register Access

```
/* Enable I2C1 clock and GPIO */
RCC->APB1ENR |= RCC_APB1ENR_I2C1EN;
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
/* PB6=SCL, PB7=SDA: open-drain, AF4, no pull (external pull-ups) */
GPIOB->MODER   |=  (2<<12)|(2<<14);   /* AF mode */
GPIOB->OTYPER  |=  (1<<6)|(1<<7);     /* open-drain */
GPIOB->AFR[0]  |=  (4<<24)|(4<<28);   /* AF4 = I2C1 */

/* Reset and configure I2C1 */
I2C1->CR1  = I2C_CR1_SWRST;  I2C1->CR1 = 0;
I2C1->CR2  = 42;              /* FREQ = APB1 clock in MHz (42 MHz) */
I2C1->CCR  = 210;             /* Standard 100 kHz: 42M/(2*100k) */
I2C1->TRISE= 43;              /* 1000ns * 42MHz / 1e9 + 1 */
I2C1->CR1 |= I2C_CR1_PE;     /* Enable peripheral */

/* Write one byte to slave */
void i2c_write(uint8_t addr, uint8_t reg, uint8_t data) {
    while (I2C1->SR2 & I2C_SR2_BUSY);
    I2C1->CR1 |= I2C_CR1_START;
    while (!(I2C1->SR1 & I2C_SR1_SB));
    I2C1->DR = addr << 1;                        /* address + write */
    while (!(I2C1->SR1 & I2C_SR1_ADDR)); (void)I2C1->SR2;
    I2C1->DR = reg;
    while (!(I2C1->SR1 & I2C_SR1_TXE));
    I2C1->DR = data;
    while (!(I2C1->SR1 & I2C_SR1_BTF));
    I2C1->CR1 |= I2C_CR1_STOP;
}
```

2.7 Complete Register Bit Definitions

I2C_CR1 — Control Register 1

Bit(s)	Name	Reset	Description
15	**SWRST**	0	Software reset. Write 1 to reset the I2C peripheral (all state machines, flags, and shift registers). Write 0 to release. Use if the bus gets stuck. PE must be 0 before setting SWRST.
13	**ALERT**	0	SMBus alert. 0: releases SMBA pin high; 1: drives SMBA low to signal an alert condition to the SMBus host.
12	**PEC**	0	Packet error checking transfer. Write 1 to append PEC byte to the next transfer. Automatically cleared by hardware after transfer.

11	POS	0	Acknowledge / PEC position. 0: ACK/PEC applies to current byte being received; 1: ACK/PEC applies to the next byte (used for 2-byte reception sequences to set NACK on the second-to-last byte).
10	ACK	0	Acknowledge enable. 0: no ACK returned after byte received; 1: ACK returned after byte received. Must be set before START for master receive. Clear before STOP to NACK last byte.
9	STOP	0	Stop generation. Write 1 in master mode to generate STOP after current byte/start completes. In slave mode: releases SCL/SDA after current byte.
8	START	0	Start generation. Write 1 to generate START condition (or Repeated START when bus is busy). Cleared by hardware after START is sent. Triggers SB flag in SR1.
7	NOSTRETCH	0	Clock stretching disable (slave mode). 0: slave stretches SCL when not ready (default); 1: clock stretching disabled — slave must always be ready. Set to 0 for most applications.
6	ENGC	0	General call enable. 0: general call address (0x00) disabled; 1: general call address generates ACK and GENCALL flag in SR2.
5	ENPEC	0	PEC enable. 0: PEC disabled; 1: packet error checking calculation enabled.
4	ENARP	0	ARP enable (SMBus). 0: ARP disabled; 1: Address Resolution Protocol enabled.
3	SMBTYPE	0	SMBus type. 0: SMBus device; 1: SMBus host. Only relevant when SMBUS=1.
1	SMBUS	0	SMBus mode. 0: I²C mode (default); 1: SMBus mode.
0	PE	0	Peripheral enable. 0: I2C peripheral disabled; 1: enabled. Must be enabled after all configuration. Disabling resets the I2C state machine.

I2C_CR2 — Control Register 2

Bit(s)	Name	Reset	Description
12	LAST	0	DMA last transfer. Set 1 before the last DMA receive transfer so that a NACK is automatically generated after the last byte.
11	DMAEN	0	DMA requests enable. 0: DMA disabled; 1: DMA request generated when TXE=1 (TX) or RXNE=1 (RX).
10	ITBUFEN	0	Buffer interrupt enable. 0: TXE/RXNE events do not generate interrupt; 1: interrupt generated on TXE=1 or RXNE=1.

9	`ITEVTEN`	0	Event interrupt enable. 0: no event interrupts; 1: interrupt on SB, ADDR, ADD10, STOPF, BTF, TXE (if ITBUFEN), RXNE (if ITBUFEN).
8	`ITERREN`	0	Error interrupt enable. 0: no error interrupts; 1: interrupt on BERR, ARLO, AF, OVR, PECERR, TIMEOUT, SMBALERT.
5:0	`FREQ[5:0]`	0	Peripheral clock frequency in MHz. Must equal the APB1 clock frequency: e.g., write 42 for 42 MHz. Range 2–50 MHz. Used internally for timing and rise-time calculations.

I2C_SR1 — Status Register 1 (read-only flags; write 0 to clear error bits)

Bit(s)	Name	Reset	Description
15	`SMBALERT`	0	SMBus alert. Set when SMBA signal is low. Cleared by writing 0.
14	`TIMEOUT`	0	Timeout or t_LOW error. Set when SCL is held low for more than 25 ms (SMBus). Cleared by writing 0.
12	`PECERR`	0	PEC error in reception. Set when received PEC does not match calculated PEC. Cleared by writing 0.
11	`OVR`	0	Overrun / underrun. In slave mode: overrun when new byte arrives before RXNE cleared (RX); underrun when clock stretching disabled and new data not ready (TX). Write 0 to clear.
10	`AF`	0	Acknowledge failure. Set when no ACK received from slave (address not recognised or NACK during data). Write 0 to clear. Generate STOP after detecting AF.
9	`ARLO`	0	Arbitration lost. Set when the master loses bus arbitration. Peripheral automatically reverts to slave mode. Write 0 to clear.
8	`BERR`	0	Bus error. Set when a misplaced START or STOP is detected. Write 0 to clear.
7	`TXE`	0	Data register empty (TX). 1: DR is empty and ready to accept next byte (TX mode). Set after each byte transmitted. Writing to DR clears TXE.
6	`RXNE`	0	Data register not empty (RX). 1: DR contains received byte ready to be read. Reading DR clears RXNE.
4	`STOPF`	0	Stop detection (slave mode). Set when STOP is detected while addressed as slave. Cleared by reading SR1 then writing to CR1.
3	`ADD10`	0	10-bit header sent (master mode). Set when the first 10-bit address byte has been sent. Master must now send the second address byte (DR = lower 8 bits of 10-bit address).
2	`BTF`	0	Byte transfer finished. Set when a byte transfer has completed AND DR has not been read (RX) or written (TX). Causes clock stretching until cleared.

1	`ADDR`	0	Address matched (slave) / sent (master). In master TX/RX: set after address byte is sent and ACK received. In slave: set when own address matches. Cleared by reading SR1 then SR2.
0	`SB`	0	Start bit generated (master mode). Set when START condition has been generated on the bus. Cleared by reading SR1 and writing to DR (the address byte).

I2C_SR2 — Status Register 2 (read-only)

Bit(s)	Name	Reset	Description
15:8	`PEC[7:0]`	0	Packet error checking register. Contains the computed PEC value of the last transfer when ENPEC=1.
7	`DUALF`	0	Dual flag (slave mode). 0: received address matched OAR1; 1: matched OAR2 (dual-address mode).
6	`SMBHOST`	0	SMBus host header. Set when address 0x08 matched (slave mode, ENARP=1).
5	`SMBDEFAULT`	0	SMBus device default address. Set when default SMBus address (0xC2) matched.
4	`GENCALL`	0	General call address. Set when general call (0x00) received and ENGC=1.
2	`TRA`	0	Transmitter / receiver. 0: data bytes being received; 1: data bytes being transmitted.
1	`BUSY`	0	Bus busy. 1: communication ongoing (START detected, not yet STOP). Prevent new master START until BUSY=0.
0	`MSL`	0	Master / slave. 0: peripheral is in slave mode; 1: master mode (set after START generated, cleared after STOP).

I2C_CCR — Clock Control Register (writable only when PE=0)

Bit(s)	Name	Reset	Description
15	`FS`	0	I²C master mode selection. 0: Standard Mode (≤100 kHz); 1: Fast Mode (≤400 kHz). Selects which CCR formula applies.
14	`DUTY`	0	Fast-mode duty cycle. Only relevant when FS=1. 0: t_LOW/t_HIGH = 2 (CCR = f_APB1/(3×f_SCL)); 1: t_LOW/t_HIGH = 16/9 (CCR = f_APB1/(25×f_SCL), for lower ripple at 400 kHz).
11:0	`CCR[11:0]`	0	Clock control value. Determines SCL period. Standard: CCR = f_APB1/(2×f_SCL). Fast (DUTY=0): CCR = f_APB1/(3×f_SCL). Fast (DUTY=1): CCR = f_APB1/(25×f_SCL). Minimum value 0x04. Set PE=0 before writing.

I2C_TRISE — Rise Time Register (writable only when PE=0)

Bit(s)	Name	Reset	Description
5:0	`TRISE[5:0]`	0	Maximum SCL rise time. Standard Mode: TRISE = floor(1000 ns × f_APB1_MHz) + 1. Fast Mode: TRISE = floor(300 ns × f_APB1_MHz) + 1. Must be set correctly for the peripheral to generate accurate timing.

I2C_FLTR — Filter Register

Bit(s)	Name	Reset	Description
4	`ANOFF`	0	Analog noise filter OFF. 0: analog filter enabled (filters glitches < ~50 ns on SCL/SDA); 1: analog filter disabled. The analog filter is ON by default and helps eliminate bus noise.
3:0	`DNF[3:0]`	0	Digital noise filter. 0: digital filter disabled; 1–15: filter enabled for spikes up to DNF × t_APB1 wide. Adds latency equal to DNF × t_APB1 to SCL/SDA inputs.

After clearing the ADDR flag (reading SR1 then SR2), the peripheral releases SCL immediately. For single-byte receive, set ACK=0 and prepare STOP before clearing ADDR to avoid a spurious extra byte being clocked in.

3. I²C on ESP32

3.1 Hardware Overview

The ESP32 integrates two I²C controllers (I2C0 and I2C1), both supporting Standard Mode (100 kHz), Fast Mode (400 kHz), and up to ~800 kHz in some configurations. Key features include: hardware command engine with a 16-entry command register FIFO, 32-byte TX and RX FIFOs, slave mode with hardware address matching, configurable SCL/SDA GPIO via IO MUX, clock stretching support, and interrupt-driven or polling operation via ESP-IDF.

The ESP32 I²C hardware uses a command-based approach — the master fills command registers (I2C_COMD0–15) with a sequence of operations (RSTART, WRITE, READ, STOP) and then triggers execution. ESP-IDF's i2c driver abstracts this into a simple API.

3.2 Key Registers — Summary

Register	Name	Key Bits / Description
I2C_SCL_LOW_PERIOD_REG	SCL Low Period	SCL_LOW_PERIOD[13:0] — SCL low period in APB clock cycles
I2C_CTR_REG	Control Register	CLK_EN, RX_LSB_FIRST, TX_LSB_FIRST, TRANS_START, MS_MODE, SCL_FORCE_OUT, SDA_FORCE_OUT
I2C_SR_REG	Status Register	SCL_STATE_LAST[2:0], MAIN_STATE[2:0], RXFIFO_CNT[5:0], TXFIFO_CNT[4:0], SLAVE_ADDRESSED, BUS_BUSY, ARB_LOST, TIME_OUT, ACK_ERR, SCLO, SDAO
I2C_TO_REG	Timeout Register	TIME_OUT[19:0] — SCL low timeout in APB cycles
I2C_SLAVE_ADDR_REG	Slave Address	ADDR_10BIT_EN, SLAVE_ADDR[14:0]
I2C_RXFIFO_ST_REG	RX FIFO Status	RXFIFO_END_ADDR[4:0], RXFIFO_START_ADDR[4:0]
I2C_FIFO_CONF_REG	FIFO Config	RX_FIFO_FULL_THRHD[4:0], TX_FIFO_EMPTY_THRHD[4:0], NONFIFO_EN, NONFIFO_RX_THRES, NONFIFO_TX_THRES, RXFIFO_FLUSH, TXFIFO_FLUSH, FIFO_ADDR_CFG_EN
I2C_DATA_APB_REG	FIFO Data (non-FIFO)	FIFO_RDATA[7:0] — byte-wide access to RX FIFO in non-FIFO mode
I2C_INT_RAW_REG	Interrupt Raw Status	TX_SEND_EMPTY_INT, RX_REC_FULL_INT, RX_FIFO_OVF_INT, END_DETECT_INT, SLAVE_TRAN_COMP_INT, ARBITRATION_LOST_INT,

		MASTER_TRAN_COMP_INT, TRANS_COMPLETE_INT, TIME_OUT_INT, TRANS_START_INT, ACK_ERR_INT, RX_REC_FULL_INT, TX_FIFO_OVF_INT, SCL_ST_TO_INT
I2C_INT_ENA_REG	Interrupt Enable	One enable bit per INT_RAW bit — 1 to enable interrupt.
I2C_INT_CLR_REG	Interrupt Clear	Write 1 to corresponding bit to clear that interrupt flag.
I2C_COMDn_REG (0–15)	Command Registers	COMMAND[13:0], COMMAND_DONE — each entry describes one bus operation
I2C_SCL_HIGH_PERIOD_REG	SCL High Period	SCL_HIGH_PERIOD[13:0] — SCL high period in APB clock cycles
I2C_SCL_RSTART_SETUP_REG	Repeated START Setup	SCL_RSTART_SETUP_TIME[9:0]
I2C_SCL_STOP_HOLD_REG	STOP Hold Time	SCL_STOP_HOLD_TIME[9:0]

3.3 Command Register Format (I2C_COMDn_REG)

Each of the 16 command registers describes one I²C bus operation. The hardware executes them sequentially when TRANS_START is set:

Field	Bits	Description
COMMAND_DONE	[31]	Read-only. Set by hardware when this command has completed execution.
op_code	[13:11]	Operation code. 000=RSTART (generate START/Repeated START); 001=WRITE (send bytes from TX FIFO + address); 010=READ (receive bytes into RX FIFO); 011=STOP (generate STOP); 100=END (marks end of command queue).
ack_check_en	[8]	ACK check enable. 1: hardware checks ACK after each byte; generates ACK_ERR interrupt if NACK received.
ack_exp	[7]	Expected ACK value. 0: expect ACK (dominant, SDA low); 1: expect NACK. Used when verifying slave response.
ack_value	[6]	ACK value sent (READ command only). 0: master sends ACK after byte received (continue reading); 1: master sends NACK after byte (last byte, stop reading).
byte_num	[7:0]	Number of bytes to send (WRITE) or receive (READ). Range 1–255. For WRITE, this includes the address byte if sent via TX FIFO.

3.4 Complete Register Bit Definitions

I2C_CTR_REG — Control Register (offset 0x004)

Bit(s)	Name	Reset	Description
8	`CLK_EN`	0	Clock gate enable for register access. 1: enable APB clock to I2C configuration registers. Must be set for register writes to take effect.
7	`RX_LSB_FIRST`	0	RX bit order. 0: MSB first (standard I²C); 1: LSB first. Non-standard — use 0 for all standard I²C devices.
6	`TX_LSB_FIRST`	0	TX bit order. 0: MSB first; 1: LSB first. Use 0 for standard I²C.
5	`TRANS_START`	0	Transfer start trigger. Write 1 to begin executing the command queue (I2C_COMDn registers). Cleared by hardware when all commands complete.
4	`MS_MODE`	0	Master / slave mode select. 0: slave mode; 1: master mode.
3	`SCL_FORCE_OUT`	1	SCL output mode. 0: open-drain (standard, requires pull-up); 1: push-pull output (only for single-master systems). Default 1; change to 0 for standard I²C multi-master.
2	`SDA_FORCE_OUT`	1	SDA output mode. 0: open-drain; 1: push-pull. Same note as SCL_FORCE_OUT.
1	`SAMPLE_SCL_LEVEL`	0	SCL sample position. 0: sample SDA on SCL low-to-high; 1: sample on SCL high level. Use 0 for standard.
0	`RIGHT_EDGE_ENABLE`	0	Right edge enable. 0: data changes on SCL rising edge; 1: changes on SCL falling edge.

I2C_SR_REG — Status Register (offset 0x008)

Bit(s)	Name	Reset	Description
30:28	`SCL_STATE_LAST[2:0]`	0	Last SCL state machine state (read-only). Debug information about SCL generation state.
26:24	`MAIN_STATE[2:0]`	0	Main I2C state machine state (read-only). 0=IDLE; 1=ADDRESS; 2=END; 3=SEND; 4=RECV; 5=WITH_DATA; 6=IDLE.
23:18	`RXFIFO_CNT[5:0]`	0	Number of bytes currently in RX FIFO (read-only). Range 0–32.
19:15	`TXFIFO_CNT[4:0]`	0	Number of bytes currently in TX FIFO (read-only). Range 0–32.
14	`SLAVE_ADDRESSED`	0	Slave addressed (slave mode, read-only). 1: own address has been matched by master.
9	`BUS_BUSY`	0	Bus busy (read-only). 1: START detected on bus, STOP not yet seen.

8	`ARB_LOST`	0	Arbitration lost (read-only). 1: master lost bus arbitration. Cleared by hardware after STOP.
6	`TIME_OUT`	0	Timeout (read-only). 1: SCL was held low for longer than I2C_TO_REG.TIME_OUT cycles.
5	`ACK_ERR`	0	ACK error (read-only). 1: slave sent NACK during address or data phase.
4	`SCLO`	0	SCL output level (read-only). Current driven level of SCL pin.
3	`SDAO`	0	SDA output level (read-only). Current driven level of SDA pin.
2	`SCL_MAIN_STATE_LAST`	0	SCL main state machine last state.
1	`SCL_EXT_CLK_LEVEL`	0	SCL external clock level (read-only). Level seen on SCL pin.
0	`SDA_MAIN_STATE_LAST`	0	SDA main state machine last state.

I2C_FIFO_CONF_REG — FIFO Configuration (offset 0x018)

Bit(s)	Name	Reset	Description
14	`NONFIFO_EN`	0	Non-FIFO mode enable. 0: data accessed via FIFO (standard DMA); 1: data accessed byte-by-byte via I2C_DATA_APB_REG.
13:9	`RX_FIFO_FULL_THRHD[4:0]`	11	RX FIFO full threshold. Interrupt INT_RAW.RX_REC_FULL_INT fires when RX FIFO depth ≥ this value. Default 11.
8:4	`TX_FIFO_EMPTY_THRHD[4:0]`	4	TX FIFO empty threshold. Interrupt INT_RAW.TX_SEND_EMPTY_INT fires when TX FIFO depth ≤ this value.
3	`RXFIFO_FLUSH`	0	Write 1 to flush (reset) the RX FIFO. Self-clearing.
2	`TXFIFO_FLUSH`	0	Write 1 to flush the TX FIFO. Self-clearing.
1	`FIFO_ADDR_CFG_EN`	0	FIFO address configuration enable. 1: enables indirect FIFO addressing.

I2C_SCL_LOW_PERIOD_REG / I2C_SCL_HIGH_PERIOD_REG

Bit(s)	Name	Reset	Description
13:0	`SCL_LOW_PERIOD[13:0]`	0	Number of APB clock cycles SCL is held low. SCL low time = SCL_LOW_PERIOD / f_APB. For 400 kHz FM with 80 MHz APB: low period = 0.6 µs → 48 cycles.
13:0	`SCL_HIGH_PERIOD[13:0]`	0	Number of APB clock cycles SCL is held high. For 400 kHz FM: high period = 0.4 µs → 32 cycles. SCL frequency = f_APB / (LOW + HIGH).

I2C_INT_RAW_REG — Interrupt Raw Register (read-only)

Bit(s)	Name	Reset	Description
13	`SCL_ST_TO_INT`	0	SCL status timeout. Set when SCL state machine is stuck.
11	`TX_FIFO_OVF_INT`	0	TX FIFO overflow. Set when a byte is written to a full TX FIFO.
9	`TRANS_COMPLETE_INT`	0	Transaction complete. Set when a STOP condition has been generated (transaction finished).
8	`MASTER_TRAN_COMP_INT`	0	Master transaction complete. Set when all command queue entries are executed.
7	`ARBITRATION_LOST_INT`	0	Arbitration lost. Set when the master detects arbitration loss.
6	`SLAVE_TRAN_COMP_INT`	0	Slave transaction complete. Set when addressed slave finishes a transaction.
5	`END_DETECT_INT`	0	END command detected. Set when hardware reaches an END op_code in the command queue, indicating the queue is done.
4	`RX_FIFO_OVF_INT`	0	RX FIFO overflow. Set when a received byte cannot be stored (FIFO full).
3	`TX_SEND_EMPTY_INT`	0	TX FIFO below threshold. Set when TX FIFO depth drops ≤ TX_FIFO_EMPTY_THRHD.
2	`TIME_OUT_INT`	0	Timeout interrupt. Set when SCL is held low beyond the TO_REG timeout.
1	`ACK_ERR_INT`	0	ACK error interrupt. Set when NACK is received when ACK was expected.
0	`RX_REC_FULL_INT`	0	RX FIFO at or above threshold. Set when RX FIFO depth ≥ RX_FIFO_FULL_THRHD.

3.5 ESP-IDF API Example

```
#include "driver/i2c.h"

#define I2C_MASTER_PORT    I2C_NUM_0
#define I2C_SCL_IO         22
#define I2C_SDA_IO         21
#define I2C_FREQ_HZ        400000   /* 400 kHz */
#define I2C_SLAVE_ADDR     0x48

void i2c_master_init(void) {
    i2c_config_t conf = {
        .mode             = I2C_MODE_MASTER,
        .sda_io_num       = I2C_SDA_IO,
        .scl_io_num       = I2C_SCL_IO,
        .sda_pullup_en    = GPIO_PULLUP_ENABLE,
        .scl_pullup_en    = GPIO_PULLUP_ENABLE,
        .master.clk_speed = I2C_FREQ_HZ,
```

```
    };
    i2c_param_config(I2C_MASTER_PORT, &conf);
    i2c_driver_install(I2C_MASTER_PORT, conf.mode, 0, 0, 0);
}

/* Write register then read back (combined transaction) */
esp_err_t i2c_reg_read(uint8_t reg, uint8_t *buf, size_t len) {
    i2c_cmd_handle_t cmd = i2c_cmd_link_create();
    i2c_master_start(cmd);
    i2c_master_write_byte(cmd, (I2C_SLAVE_ADDR << 1) | I2C_MASTER_WRITE, true);
    i2c_master_write_byte(cmd, reg, true);
    i2c_master_start(cmd);                              /* Repeated START */
    i2c_master_write_byte(cmd, (I2C_SLAVE_ADDR << 1) | I2C_MASTER_READ, true);
    if (len > 1) i2c_master_read(cmd, buf, len-1, I2C_MASTER_ACK);
    i2c_master_read_byte(cmd, buf+len-1, I2C_MASTER_NACK);
    i2c_master_stop(cmd);
    esp_err_t ret = i2c_master_cmd_begin(I2C_MASTER_PORT, cmd, pdMS_TO_TICKS(100));
    i2c_cmd_link_delete(cmd);
    return ret;
}
```

> *Always send NACK on the last byte of a read transaction, followed by STOP. Sending ACK after the last byte signals the slave to prepare another byte, causing bus contention.*

4. I²C on S32K1 (LPI2C)

4.1 Hardware Overview

S32K1xx devices feature the Low Power Inter-Integrated Circuit (LPI2C) peripheral, derived from the i.MX RT I²C architecture. LPI2C supports Standard (100 kHz), Fast (400 kHz), and Fast-Plus (1 MHz) modes, along with ultra-fast mode (no ACK, single-direction). Key features: configurable glitch filter, programmable setup/hold times, 4-word TX/RX FIFOs with watermarks, DMA support, host request input, and operation in all low-power modes. LPI2C requires module enable (MEN=0 before most config changes, similar to LPSPI).

4.2 Key Registers — Summary

Register	Name	Key Bits
LPI2C_MCR	Master Control Register	MEN, RST, DOZEN, DBGEN, RTF (reset TX FIFO), RRF (reset RX FIFO)
LPI2C_MSR	Master Status Register	BBF (bus busy), MBF (master busy), DMF, PLTF, FEF, ALF, NDF, SDF, EPF, RDF, TDF
LPI2C_MIER	Master Interrupt Enable	DMAIE, PLTIE, FEIE, ALIE, NDIE, SDIE, EPIE, RDIE, TDIE
LPI2C_MDER	Master DMA Enable	RDDE (RX DMA), TDDE (TX DMA)
LPI2C_MFCR	Master FIFO Control	RXWATER[1:0], TXWATER[1:0] — FIFO watermark thresholds
LPI2C_MFSR	Master FIFO Status	RXCOUNT[2:0], TXCOUNT[2:0] — current FIFO fill level
LPI2C_MTDR	Master Transmit Data Register	CMD[2:0], DATA[7:0] — command + data byte combined
LPI2C_MRDR	Master Receive Data Register	RXEMPTY, DATA[7:0] — received byte
LPI2C_MCCR0	Master Clock Config 0 (normal)	CLKHI[5:0], CLKLO[5:0], SETHOLD[5:0], DATAVD[5:0]
LPI2C_MCCR1	Master Clock Config 1 (HS/UFm)	Same fields as MCCR0 but for High-Speed or Ultra-Fast mode
LPI2C_MCFGR0	Master Config 0	RDMO, CIRFIFO, HRSEL, HRPOL, HREN
LPI2C_MCFGR1	Master Config 1	PINCFG[2:0], MATCFG[2:0], TIMECFG, IGNACK, AUTOSTOP, PRESCALE[2:0]
LPI2C_MCFGR2	Master Config 2	FILTSDA[3:0], FILTSCL[3:0], BUSIDLE[11:0]
LPI2C_MCFGR3	Master Config 3	PINLOW[11:0] — pin low timeout

4.3 Master Transmit Data Register — CMD Field

LPI2C uses a combined CMD+DATA write to MTDR to control bus operations. The CMD field selects the action:

CMD[2:0]	Operation	DATA field usage
000	Transmit DATA[7:0]	Byte written to TX FIFO and shifted onto SDA. Slave must ACK.
001	Receive (DATA+1) bytes	DATA[7:0]+1 = number of bytes to clock in. Received bytes stored in RX FIFO.
010	Generate STOP	DATA ignored. Generates STOP condition and ends transaction.
011	Receive and discard	Clock in DATA+1 bytes but discard them (no store in FIFO).
100	Generate START + send address (7-bit write)	DATA[7:1]=address, DATA[0]=0. Generates START then address+W.
101	Generate START + send address (7-bit read)	DATA[7:1]=address, DATA[0]=1. Generates START then address+R.
110	Generate START + send address (10-bit write)	First byte: 11110xx0 (high bits). Second MTDR write sends low 8 bits.
111	Generate START + send address (10-bit read)	Same as 110 but read direction.

4.4 Complete Register Bit Definitions

LPI2C_MCR — Master Control Register

Bit(s)	Name	Reset	Description
9	`DBGEN`	0	Debug enable. 0: LPI2C halted in debug mode; 1: continues operation during debug halt.
8	`DOZEN`	0	Doze mode enable. 0: LPI2C operates in doze mode; 1: disabled in doze mode (low power).
3	`RRF`	0	Reset receive FIFO. Write 1 to flush the RX FIFO. Self-clearing.
2	`RTF`	0	Reset transmit FIFO. Write 1 to flush the TX FIFO. Self-clearing.
1	`RST`	0	Software reset. Write 1 to reset all internal logic. Self-clearing. MEN must be 0 first.

0	MEN	0	Master enable. 0: LPI2C master disabled; 1: enabled. Configuration must be complete before setting MEN=1.

LPI2C_MSR — Master Status Register

Bit(s)	Name	Reset	Description
25	BBF	0	Bus busy flag (read-only). 0: bus is idle; 1: START detected, STOP not yet seen.
24	MBF	0	Master busy flag (read-only). 0: master is idle; 1: master is actively controlling the bus.
13	DMF	0	Data match flag. Set when received data matches LPI2C_MDMR value (if data match configured). Write 1 to clear.
12	PLTF	0	Pin low timeout flag. Set when SCL or SDA has been continuously low for longer than MCFGR3.PINLOW cycles. Write 1 to clear.
11	FEF	0	FIFO error flag. Set when TX FIFO underflows (master transmit underrun) or RX FIFO overflows. Write 1 to clear.
10	ALF	0	Arbitration lost flag. Set when the master loses arbitration. The master automatically stops transmitting. Write 1 to clear.
9	NDF	0	NACK detect flag. Set when a NACK is received from the slave (either on address or data byte). Write 1 to clear. Action depends on MCFGR1.IGNACK.
8	SDF	0	STOP detect flag. Set when a STOP condition is detected on the bus. Write 1 to clear.
7	EPF	0	End packet flag. Set when a STOP or Repeated START is detected. Write 1 to clear.
1	RDF	0	Receive data flag. 1: RX FIFO contains data at or above RXWATER watermark. Drives RX interrupt or DMA request.
0	TDF	1	Transmit data flag. 1: TX FIFO at or below TXWATER watermark — ready for more data. Default 1. Drives TX interrupt or DMA.

LPI2C_MCCR0 — Master Clock Configuration Register 0 (normal mode)

Bit(s)	Name	Reset	Description
29:24	CLKHI[5:0]	0	SCL high period in functional clock cycles. Controls t_HIGH of SCL. Value must meet I²C spec minimum high time for the target speed.
21:16	CLKLO[5:0]	0	SCL low period in functional clock cycles. Controls t_LOW of SCL. t_LOW ≥ CLKLO / f_LPSPI_CLK.
13:8	SETHOLD[5:0]	0	Setup and hold delay. Controls SDA setup time after SCL rising edge and SDA hold time after SCL falling edge.

5:0	DATAVD[5:0]	0	Data valid delay. Minimum SCL low time before SDA changes (data valid window). Ensures SDA is stable before SCL rises.

LPI2C_MCFGR1 — Master Configuration Register 1

Bit(s)	Name	Reset	Description
26:24	PINCFG[2:0]	0	Pin configuration. 000: 2-pin open-drain (standard); 001: 2-pin output only (no filtering); 010: 2-pin push-pull; 011: 4-pin push-pull; 100: 2-pin open-drain with separate pin; 101–111: reserved.
18:16	MATCFG[2:0]	0	Match configuration. 000: data match disabled; 001: match address 0; 010: match first/second received data byte; 011–111: various range and combined matches for data-triggered DMA.
10	TIMECFG	0	Timeout configuration. 0: pin low timeout measures SCL+SDA combined; 1: measures SCL and SDA independently.
9	IGNACK	0	Ignore NACK. 0: NACK generates NDF flag and aborts transfer; 1: NACK is ignored, transfer continues. Use with care — non-standard behaviour.
8	AUTOSTOP	0	Automatic STOP generation. 0: STOP only on explicit CMD=010; 1: STOP generated automatically when TX FIFO empties.
2:0	PRESCALE[2:0]	0	Clock prescaler. LPI2C functional clock = peripheral clock / 2^PRESCALE. 000=÷1, 001=÷2, ..., 111=÷128. Sets the TQ resolution for MCCR0.

LPI2C_MCFGR2 — Master Configuration Register 2

Bit(s)	Name	Reset	Description
27:16	BUSIDLE[11:0]	0	Bus idle timeout. Number of functional clock cycles before the bus is considered idle after last STOP. If 0, timeout is disabled.
11:8	FILTSDA[3:0]	0	SDA glitch filter cycles. Input glitches shorter than FILTSDA+1 functional clock cycles are filtered out.
3:0	FILTSCL[3:0]	0	SCL glitch filter cycles. Input glitches shorter than FILTSCL+1 functional clock cycles are filtered out on SCL.

4.5 SDK Initialization and Transfer Example

```
#include "lpi2c_master_driver.h"

lpi2c_master_state_t i2cState;

const lpi2c_master_user_config_t i2cConfig = {
    .slaveAddress          = 0x48U,
```

```
    .is10bitAddr          = false,
    .baudRate             = 400000U,   /* 400 kHz */
    .operatingMode        = LPI2C_FAST_MODE,
    .transferType         = LPI2C_USING_INTERRUPTS,
    .rxDMAChannel         = 255U,
    .txDMAChannel         = 255U,
    .masterCallback       = NULL,
    .callbackParam        = NULL,
};
LPI2C_DRV_MasterInit(INST_LPI2C1, &i2cConfig, &i2cState);

/* Write 1 byte register address, then read 2 bytes */
uint8_t txBuf[1] = { 0x00 };   /* register address */
uint8_t rxBuf[2];

/* Send register address */
LPI2C_DRV_MasterSendDataBlocking(INST_LPI2C1, txBuf, 1U, false, 100U);
/* Read 2 bytes (false = no STOP between, true = repeated START) */
LPI2C_DRV_MasterReceiveDataBlocking(INST_LPI2C1, rxBuf, 2U, true, 100U);
```

LPI2C requires MEN=0 before modifying MCCR0, MCFGR0/1/2/3, and MIER. Always disable, configure, then re-enable. Flush FIFOs (RTF, RRF) before starting a new transaction if previous errors occurred.

5. I²C in AUTOSAR

5.1 AUTOSAR I2C Architecture Overview

AUTOSAR Classic Platform does not define a standardised I²C Handler/Driver module in the same way as SPI (Spi module) or CAN (Can module) in releases up to R20-11. I²C support is typically provided through vendor-specific MCAL extensions or Complex Device Drivers (CDDs). However, the AUTOSAR Adaptive Platform (AP) provides a standardised I2C binding, and many Classic Platform toolchains (EB tresos, Vector DaVinci) include an I2c MCAL module following the SPI driver pattern.

The typical AUTOSAR I²C integration follows this layering:

Layer	Module	Responsibility
Application / RTE	SWC / RTE	Reads/writes data via I²C-mapped sensor interfaces or direct CDD calls.
Complex Device Driver	CDD_I2c	Wraps vendor I2c MCAL; provides application-level read/write functions for specific sensors or actuators.
MCAL	I2c Driver	Hardware register access; channel/job/sequence abstraction (similar to SPI MCAL); interrupt/DMA management.
ECU Abstraction	IoHwAb	Maps logical signals to physical I²C addresses and registers.

5.2 I2c MCAL Channel/Job/Sequence Abstraction

When vendors implement an AUTOSAR-style I2c driver, they typically mirror the SPI module pattern:

- Channel — a data buffer entity with a defined direction (TX, RX, or TX+RX), address, and register offset.
- Job — one or more channels transferred to/from the same I²C slave address on the same hardware unit (I2C0, I2C1, etc.).
- Sequence — an ordered list of jobs. Async sequences are scheduled by the BSW scheduler; sync sequences block the caller.

Config Field	Type	Full Description
I2cChannelId	uint8	Unique channel identifier. Used in all I2c API calls.
I2cSlaveAddress	uint8 / uint16	7-bit (0x00–0x7F) or 10-bit slave I²C address. Must match device pull-up configuration.
I2cAddressingMode	7BIT / 10BIT	Selects 7-bit or 10-bit slave addressing format.
I2cDirection	TX / RX / TXRX	TX: master sends data. RX: master receives data. TXRX: combined write-then-read (register read).
I2cRegisterAddress	uint8 / uint16	Internal device register address sent before the read phase in TXRX transactions.

I2cDataLength	uint16	Number of data bytes to transfer. For RX, this is how many bytes to clock from the slave.
I2cBaudrate	uint32 (Hz)	I²C clock speed. Driver selects nearest hardware prescaler. Typical: 100000, 400000, 1000000.
I2cChannelType	IB / EB	Internal Buffer or External Buffer (same semantics as SPI MCAL).
I2cTransferType	INTERRUPT / DMA	Selects interrupt-driven or DMA-driven transfer for this channel.

5.3 I2c MCAL API Functions

Function	Description	Return
I2c_Init(ConfigPtr)	Initialise I²C hardware and internal state.	void
I2c_DeInit()	De-initialise I²C hardware.	Std_ReturnType
I2c_WriteIB(Channel, DataBufferPtr)	Write data to internal buffer of channel (IB type).	Std_ReturnType
I2c_ReadIB(Channel, DataBufferPtr)	Read received data from internal buffer.	Std_ReturnType
I2c_SetupEB(Channel, SrcPtr, DesPtr, Len)	Assign external TX/RX buffers to channel (EB type).	Std_ReturnType
I2c_AsyncTransmit(Sequence)	Start asynchronous transfer of a sequence.	Std_ReturnType
I2c_SyncTransmit(Sequence)	Start synchronous (blocking) transfer of a sequence.	Std_ReturnType
I2c_GetSequenceResult(Sequence)	Get result of last sequence: OK, PENDING, FAILED, CANCELLED.	I2c_SeqResultType
I2c_GetStatus()	Get overall driver status: UNINIT, IDLE, BUSY.	I2c_StatusType
I2c_MainFunction()	Polling function called by BSW scheduler (non-interrupt mode).	void

5.4 AUTOSAR I2c Configuration Snippet

```
/* Generated I2c_PBCfg.c — AUTOSAR I2c MCAL Configuration */

static const I2c_ChannelConfigType I2c_ChannelConfig[] = {
    {
        .I2cChannelId         = I2cConf_I2cChannel_TempSensor,
        .I2cSlaveAddress      = 0x48U,         /* TMP102 address */
        .I2cAddressingMode    = I2C_7BIT_ADDR,
```

```
        .I2cDirection         = I2C_TX_RX,     /* write reg, then read */
        .I2cRegisterAddress   = 0x00U,         /* temperature register */
        .I2cDataLength        = 2U,            /* 16-bit temperature */
        .I2cChannelType       = I2C_IB,
        .I2cDefaultData       = 0x00U,
    },
};

static const I2c_JobConfigType I2c_JobConfig[] = {
    {
        .I2cJobId             = I2cConf_I2cJob_ReadTemperature,
        .I2cHwUnit            = I2cConf_I2cHwUnit_I2C0,
        .I2cBaudrate          = 400000U,       /* 400 kHz */
        .I2cJobEndNotification= NULL_PTR,
        .ChannelList          = { I2cConf_I2cChannel_TempSensor },
        .ChannelCount         = 1U,
    },
};

/* Usage */
I2c_AsyncTransmit(I2cConf_I2cSequence_ReadTemp);
while(I2c_GetSequenceResult(I2cConf_I2cSequence_ReadTemp)==I2C_SEQ_PENDING);
I2c_ReadIB(I2cConf_I2cChannel_TempSensor, rxData);
```

> *AUTOSAR Classic does not standardise I2c across all vendors. When integrating vendor MCAL I2c, verify the module ID, API names, and configuration schema against the vendor SWS document. Use IoHwAb to isolate application SWCs from direct MCAL calls.*

6. I²C on Kinetis KV5x (I2C)

6.1 Hardware Overview

The Kinetis KV5x family (KV56F, KV58F) includes up to 2 I²C modules (I2C0, I2C1). The peripheral supports Standard (100 kHz) and Fast (400 kHz) modes, 7-bit and 10-bit addressing, glitch filter, wakeup from stop mode, SMBus compatibility, broadcast general call, DMA support, and timeout detection. The module uses a register-based approach with a single data register (I2Cx_D) and a dedicated status register.

6.2 Key Registers — Summary

Register	Name	Key Bits
I2Cx_A1	Address Register 1	AD[7:1] — 7-bit slave address (bits [7:1], bit 0 unused)
I2Cx_F	Frequency Divider Register	MULT[1:0], ICR[5:0] — sets I²C clock speed from bus clock
I2Cx_C1	Control Register 1	IICEN, IICIE, MST, TX, TXAK, RSTA, WUEN, DMAEN
I2Cx_S	Status Register	TCF, IAAS, BUSY, ARBL, RAM, SRW, IICIF, RXAK
I2Cx_D	Data I/O Register	DATA[7:0] — write to transmit, read to receive (auto-starts transfer)
I2Cx_C2	Control Register 2	GCAEN, ADEXT, HDRS, SBRC, RMEN, AD[10:8] — 10-bit address extension
I2Cx_FLT	Programmable Input Glitch Filter	STARTF, SSIE, STOPF, SHEN, FLT[3:0] — filter width and START/STOP detection
I2Cx_RA	Range Address Register	RAD[7:1] — range address (accept range of addresses in slave mode)
I2Cx_SMB	SMBus Control and Status	FACK, ALERTEN, SIICAEN, TCKSEL, SLTF, SHTF1, SHTF2, SHTF2IE
I2Cx_A2	Address Register 2	SAD[7:1] — SMBus address
I2Cx_SLTH	SCL Low Timeout Register H	SSLT[15:8] — high byte of SCL low timeout counter
I2Cx_SLTL	SCL Low Timeout Register L	SSLT[7:0] — low byte of SCL low timeout counter

6.3 Frequency Divider (I2Cx_F) — ICR Table

The I2C clock speed is set by ICR[5:0] and MULT[1:0]. The SCL divider = MULT × ICR_divider, where ICR_divider is a lookup value:

```
/* SCL frequency = Bus clock / (MULT * ICR_divider)
   MULT: 00=1x, 01=2x, 10=4x
   ICR examples (from reference manual Table):
   ICR=0x00 => divider=20,  ICR=0x05 => divider=48,  ICR=0x10 => divider=160
   ICR=0x1C => divider=288, ICR=0x20 => divider=384, ICR=0x27 => divider=1920

/* Example: Bus=60MHz, target=400kHz, MULT=01(2x)
   Required divider = 60MHz / (2 * 400kHz) = 75
   Closest ICR: ICR=0x0D => divider=80 => 60M/(2*80)=375kHz

/* Example: Bus=60MHz, target=100kHz, MULT=10(4x)
   Required divider = 60MHz / (4 * 100kHz) = 150
   ICR=0x1A => divider=160 => 60M/(4*160)=93.75kHz */
```

6.4 Complete Register Bit Definitions

I2Cx_C1 — Control Register 1

Bit(s)	Name	Reset	Description
7	**IICEN**	0	I²C enable. 0: module disabled (pins released); 1: module enabled. Set last after all configuration.
6	**IICIE**	0	I²C interrupt enable. 0: interrupt disabled; 1: interrupt generated when IICIF=1 in the Status Register.
5	**MST**	0	Master mode select. 0: slave mode; 1: master mode. Writing 0→1 generates a START condition; writing 1→0 generates a STOP.
4	**TX**	0	Transmit mode select. 0: receive mode; 1: transmit mode. Must be set to 1 before sending address or data, cleared before reading received data.
3	**TXAK**	0	Transmit acknowledge enable. 0: send ACK after received byte (continue reception); 1: send NACK after received byte. Set TXAK=1 before reading the second-to-last byte to NACK the last byte.
2	**RSTA**	0	Repeat START. Write 1 to generate a Repeated START condition without releasing the bus. Self-clearing. Used to change direction (write→read) in combined transactions.
1	**WUEN**	0	Wake-up enable. 0: no wake from stop mode; 1: match on own address wakes MCU from stop mode.
0	**DMAEN**	0	DMA enable. 0: DMA disabled; 1: DMA requests generated on data register events (IICIF-related). Use with DMA controller for large transfers.

I2Cx_S — Status Register

Bit(s)	Name	Reset	Description

7	`TCF`	1	Transfer complete flag (read-only). 0: transfer in progress; 1: byte transfer complete. Cleared by reading DR (receive) or writing to DR (transmit). Used to pace byte-by-byte transfers.
6	`IAAS`	0	Addressed as a slave. Set when own address matches received address in slave mode. Clear by reading C1 register after checking SRW bit.
5	`BUSY`	0	Bus busy (read-only). 0: bus is idle (STOP detected or never used); 1: bus is busy (START detected, no STOP). Wait for BUSY=0 before generating START in multi-master systems.
4	`ARBL`	0	Arbitration lost. Set when master loses arbitration. Must be cleared by writing 1. Module automatically switches to slave mode.
3	`RAM`	0	Range address match. Set when a received address falls within the range address (I2Cx_A1 to I2Cx_RA) in slave mode.
2	`SRW`	0	Slave read/write (slave mode, read-only). 0: slave receive, master writing; 1: slave transmit, master reading. Read after IAAS=1 to determine direction.
1	`IICIF`	0	I²C interrupt flag. Set after byte transfer, address match, arbitration loss, or START/STOP detection (per filter config). Write 1 to clear. Drives interrupt if IICIE=1.
0	`RXAK`	0	Receive acknowledge (read-only). 0: ACK received from slave (success); 1: NACK received. Read after writing to DR (after TCF=1) to verify slave acknowledged.

I2Cx_C2 — Control Register 2

Bit(s)	Name	Reset	Description
7	`GCAEN`	0	General call address enable. 0: general call (0x00) ignored; 1: general call address generates IAAS and interrupt.
6	`ADEXT`	0	Address extension. 0: 7-bit address (I2Cx_A1[7:1]); 1: 10-bit address enabled (upper 3 bits from C2.AD[2:0], lower 8 from A1[7:0] — note this differs from standard 10-bit framing).
5	`HDRS`	0	High drive select. 0: normal drive strength on SDA/SCL; 1: high drive strength (for long cables or high capacitance buses). Increases current sink capability.
4	`SBRC`	0	Slave baud rate control. 0: slave clock rate follows master; 1: independent slave baud rate (for very low-power slave timing).
3	`RMEN`	0	Range address matching enable. 0: disabled; 1: address matching in range I2Cx_A1 to I2Cx_RA generates IAAS.
2:0	`AD[10:8]`	0	10-bit address extension bits [10:8]. Used with ADEXT=1 and I2Cx_A1 for 10-bit slave addressing.

I2Cx_FLT — Programmable Input Glitch Filter Register

Bit(s)	Name	Reset	Description
7	**STARTF**	0	I²C bus START detect flag. Set when a START condition is detected on the bus. Write 1 to clear. Generates interrupt if SSIE=1.
6	**SSIE**	0	I²C bus stop or START interrupt enable. 1: interrupt generated when STARTF or STOPF is set.
5	**STOPF**	0	I²C bus STOP detect flag. Set when a STOP condition is detected. Write 1 to clear.
4	**SHEN**	0	Stop hold enable. 0: STOP detection immediate; 1: module waits one clock after STOP before allowing new START. Prevents false re-triggering.
3:0	**FLT[3:0]**	0	Input glitch filter. 0: filter disabled; 1–15: filter enabled — glitches shorter than FLT × bus clock periods are suppressed on SCL and SDA inputs.

6.5 MCUXpresso SDK Example

```
#include "fsl_i2c.h"

i2c_master_config_t masterConfig;
I2C_MasterGetDefaultConfig(&masterConfig);
masterConfig.baudRate_Bps = 400000U;  /* 400 kHz */
I2C_MasterInit(I2C0, &masterConfig, CLOCK_GetFreq(kCLOCK_BusClk));

/* Write register + data */
uint8_t txBuf[2] = { 0x01, 0xFF };  /* config register, value */
i2c_master_transfer_t xfer;
xfer.slaveAddress   = 0x48U;
xfer.direction      = kI2C_Write;
xfer.subaddress     = 0;
xfer.subaddressSize = 0;
xfer.data           = txBuf;
xfer.dataSize       = 2U;
xfer.flags          = kI2C_TransferDefaultFlag;
I2C_MasterTransferBlocking(I2C0, &xfer);

/* Read 2 bytes from register 0x00 */
uint8_t rxBuf[2];
xfer.direction      = kI2C_Read;
xfer.subaddress     = 0x00U;
xfer.subaddressSize = 1U;
xfer.data           = rxBuf;
xfer.dataSize       = 2U;
I2C_MasterTransferBlocking(I2C0, &xfer);
```

On KV5x, after detecting RXAK=1 (NACK), generate a STOP condition immediately (write MST=0 or STOP bit). Failing to do so may leave the bus in an undefined state.

7. Platform Comparison & Best Practices

7.1 I²C Peripheral Feature Comparison

Feature	STM32F4 I2C	ESP32 I2C	S32K1 LPI2C	AUTOSAR I2c	KV5x I2C
Max Speed	400 kHz (FM)	~800 kHz	1 MHz (FM+)	Platform dep.	400 kHz (FM)
10-bit Address	Yes	Yes	Yes	Config	Yes (C2.ADEXT)
Clock Stretching	Yes (NOSTRETCH)	Yes	Yes	Config	Yes (auto)
DMA Support	Yes	No (HW FIFO)	Yes (EDMA)	Yes	Yes
Glitch Filter	Analog + Digital	Digital	Digital (FILTS)	Config	Digital (FLT)
SMBus Mode	Yes (SMBUS bit)	No	No	No	Yes (SMB reg)
FIFO	None (1 DR)	32-byte TX/RX	4-word TX/RX	SW managed	None (1 DR)
Slave Mode	Yes	Yes	Yes	Yes	Yes
Multi-Master	Yes (ARLO)	Yes	Yes (ALF)	Yes	Yes (ARBL)
Low-Power Wakeup	No	No	Yes (DOZEN)	Config	Yes (WUEN)

7.2 Pull-up Resistor Selection Guide

Speed	Bus Voltage	Bus Capacitance	Recommended Pull-up	Formula
Standard (100 kHz)	3.3 V	< 100 pF	4.7 kΩ	R = t_r / (0.8473 × C_bus); t_r ≤ 1000 ns
Standard (100 kHz)	5.0 V	< 200 pF	2.2 kΩ	t_r max 1000 ns
Fast (400 kHz)	3.3 V	< 100 pF	2.2 kΩ	t_r max 300 ns
Fast (400 kHz)	3.3 V	< 400 pF	1.0 kΩ	Near minimum; check I/O sink current

Fast-Plus (1 MHz)	3.3 V	< 100 pF	560–1000 Ω	FM+ devices have higher sink current

7.3 Common Pitfalls and Debugging

Symptom	Root Cause	Fix
Bus stuck low (SDA or SCL)	Slave clock-stretching or crashed mid-transaction	Toggle SCL 9 times manually to free stuck slave; generate STOP.
NACK on every address	Wrong slave address, missing pull-ups, or device not powered	Verify address with I²C scanner; check Vcc and pull-up resistors.
Spurious extra byte received	ACK not cleared before STOPF (STM32F4)	Set ACK=0 and STOP=1 before reading last byte; clear ADDR carefully.
STM32 ADDR never clears	SR1 read but SR2 not read afterward	Clear ADDR by reading SR1 then SR2 in sequence — both reads required.
Garbled data at 400 kHz	Pull-ups too large, bus capacitance too high	Reduce pull-up value or add bus buffer (e.g., PCA9306); shorten wires.
ESP32 ACK_ERR_INT immediately	Device at wrong address or not responding	Run I²C scan; verify hardware; check SDA/SCL GPIO assignment.
LPI2C NDF flag, transfer aborts	Slave NACK (normal for broadcast/address scan)	Set IGNACK=1 for scanning; for real devices: verify address and Vcc.
KV5x ARBL set on first transfer	Another master generated START at same time	Wait for BUSY=0 before START; implement back-off on ARBL.
AUTOSAR I2c_AsyncTransmit fails	Sequence already PENDING or ISR not configured	Check I2c_GetStatus() == I2C_IDLE; verify OS ISR routing to I2c_Isr.

7.4 I²C Best Practices

- Always place 100 nF decoupling capacitors close to every I²C device's Vcc pin to prevent noise coupling onto SDA/SCL.
- Use the minimum pull-up resistor that satisfies rise-time requirements — unnecessarily high values waste current; too low causes excessive sink current.
- Keep I²C trace length short (< 30 cm for Fast Mode). Use I²C buffers or expanders (PCA9306, PCA9515) for longer runs.
- Implement a bus recovery routine (9 SCL pulses + START + STOP) for stuck-bus conditions in production firmware.
- In multi-master systems, always check BUSY before generating START and handle arbitration loss (ARBL/ALF) by backing off and retrying.
- Use clock stretching only when necessary — some masters have limited stretching support (e.g., ESP32 master has a maximum timeout).
- For 10-bit addressing, ensure all masters and slaves on the bus support 10-bit mode before enabling it.

- In AUTOSAR, use IoHwAb to map I²C sensor reads to runnables; never call I2c MCAL directly from application SWCs.

8. Appendix

8.1 GPIO and Pin Configuration Quick Reference

Platform	SDA Pin	SCL Pin	Config Method	Mode Required
STM32F4 I2C1	PB7 (AF4)	PB6 (AF4)	MODER=AF, OTYPER=OD, AFR=4	Open-drain, no internal pull (use external)
STM32F4 I2C2	PB11 (AF4)	PB10 (AF4)	MODER=AF, OTYPER=OD, AFR=4	Open-drain
STM32F4 I2C3	PC9 (AF4)	PA8 (AF4)	MODER=AF, OTYPER=OD, AFR=4	Open-drain
ESP32	Any GPIO	Any GPIO	i2c_config_t sda/scl_io_num	Internal pull-up via sda/scl_pullup_en
S32K1 LPI2C0	PTB1	PTB0	PINS_DRV_Init, open-drain	Open-drain (ODE=1 in PCR)
KV5x I2C0	PTD9 (ALT2)	PTD8 (ALT2)	PORT_PCR mux=2, ODE=1	Open-drain (ODE bit in PORT_PCR)

For STM32F4 I²C, SDA and SCL GPIO pins MUST be configured as open-drain (OTYPER bit = 1). Using push-pull mode will fight against the external pull-up resistor and damage the I/O or the slave device.

8.2 Reserved I²C Addresses (7-bit Mode)

Address(es)	Reason Reserved	Notes
0x00	General Call address	Broadcast to all slaves. Data byte 0x04 = reset; 0x06 = reset + write address.
0x01	CBUS address	Legacy CBUS protocol. Avoid.
0x02	Reserved	Future use.
0x03	Reserved	Future use.
0x04–0x07	Reserved	High-Speed mode master code (0x00001xx).
0x78–0x7B	10-bit address prefix	Frames 11110xx indicate 10-bit addressing when seen as first byte.
0x7C–0x7F	Reserved	Device ID (0x7C), future use (0x7D–0x7F).

8.3 Common I²C Device Addresses

Device Type	Typical Address(es)	Notes
TMP102 Temperature Sensor	0x48–0x4B	ADD0 pin: GND=0x48, Vcc=0x49, SDA=0x4A, SCL=0x4B
DS3231 RTC	0x68	Fixed address, single device per bus
AT24C EEPROM	0x50–0x57	A0–A2 pins select address within range
MPU-6050 IMU	0x68 / 0x69	AD0 pin: GND=0x68, Vcc=0x69
BMP280 Pressure Sensor	0x76 / 0x77	SDO pin: GND=0x76, Vcc=0x77
SSD1306 OLED Display	0x3C / 0x3D	SA0 pin or solder bridge selects address
PCA9685 PWM Controller	0x40–0x7F	A0–A5 pins; all-call 0x70
MCP4725 12-bit DAC	0x60 / 0x61	A0 pin selects address

8.4 References

- STM32F4xx Reference Manual RM0090, Chapter: Inter-integrated circuit (I²C) interface — STMicroelectronics
- ESP-IDF Programming Guide: I2C Driver — Espressif Systems
- S32K1xx Reference Manual S32K1XXRM, Chapter: Low Power Inter-Integrated Circuit (LPI2C) — NXP Semiconductors
- AUTOSAR SWS SPIHandlerDriver (vendor I2c modules follow this pattern) — AUTOSAR Consortium
- KV5x Reference Manual KV5XP144M240RM, Chapter: Inter-Integrated Circuit (I2C) — NXP Semiconductors
- NXP Application Note AN10216: I²C Manual — NXP Semiconductors
- I²C-bus specification and user manual UM10204 Rev. 7 — NXP Semiconductors
- SMBus Specification Version 3.1 — System Management Bus Working Group